Introduction to computer programming (2023/2024)

Paolo Franciosa (credits to Massimo Lauria)

Exam January 22, 2024

Working methods: The exam text, the test files of the exercises and a synoptic table on Python syntax can be found in the `exam` folder on your desk, in the "Statistica-Laboratorio" virtual machine. The solutions to the exercises must be placed in the same folder `exam`. **First compile the file** `studente.txt` with the first name, surname and matriculation information

```
nome: Giacomo
cognome: Leopardi
matricola: 123456789
```

Not completing the `student.txt` file is like not handing in the assignment. For the exam, you can only have

- your ID or passport;
- beverages and snacks;

and you must leave your bags and backpacks in the corner of the room, with mobile phones and electronic devices switched off inside.

The grade is the sum of the points of the exercises, so it is possible to get 30 even without completing them all.

Use of test files: tests check very briefly the setting of the solutions and passing the tests does not at all imply that the solution is correct. Read the text of the exercises before starting to solve them. To complete each exercise exercise you must

- write a solution file **with the specified name**;
- have the function **with the specified name** inside;
- place the solution file in the `exam` folder;

- run the `python3 <testfile>` command in that folder

where `<testfile>` should of course be replaced with the name of the appropriate test file for the exercise you are working on.

There is an independent test file for each exercise, so you can work on the exercises one at a time with more ease. The result of each test is a screen (or several screens) showing

- whether the file with the correct name was found,
- whether the file contains the function with the correct name,
- whether calls to the function with different parameter values end by returning correct results.

For each function written, calls are made with different values of the parameters. The result of the tests is returned with the character

- `E` if the call cannot be executed,
- `F` if the function does not return the correct result,
- `.` if the function returns the correct result.

1 Rectangle area (1 point)

Write a function `rectangle_area(s_1, s_2)` that:

- takes two non-negative number `s_1` `s_2` as parameters;
- returns the area of a rectangle with sides' measures `s_1` and `s_2`.

The Python function must be saved in file: `rectangle_area.py`

Test file: `test_rectangle_area.py`

2 Sum_negatives (1 point)

Write a function `sum_negatives(L)` that,

- takes a list `L` as parameter, in which each element is a number (`int` or `float`);
- returns the sum of all negative numbers in the list `L`. You can assume that `L` is a list of numbers (each element is either `int` or `float`).

```
a = [3, -1, 0, -4, 7]
print(sum_negatives(a))
```

```
-5
```

The Python function must be saved in file: `sum_negatives.py`

Test file: `test_sum_negatives.py`

3 Almost equal lists (1 point)

Write a function `almost_equal(a, b)` that:

- takes two non-empty lists `a`, `b` with the same length as parameters, where each element is a number (`int` or `float`);
- returns `True` if all pairs of elements in `a` and `b` differ by at most 1 (in absolute value), `False` otherwise.

You can **assume** that the lists are not empty, that they contain only numbers and that they have the same length.

```
x = [7, 5, 0, 5]
y = [6, 5, -1, 6]
z = [5, 4, -2, 3]
k = [5, 4, -1, -3]
print(almost_equal(x, y))
print(almost_equal(x, z))
print(almost_equal(y, z))
print(almost_equal(z, k))
```

```
True
False
False
False
```

The Python function must be saved in file: `almost_equal.py`

Test file: `test_almost_equal.py`

4 Create doubled dictionary (1 point)

Write a function `dict_double(d)` that:

- takes as arguments a dictionary `d` with values of type `int` or `float`;
- returns a new dictionary containing, for each key in `d`, an item with the same key and a value that is twice the value in `d`.

You may assume that alle values are numeric.

For example:

```
d = {"Mario": 1000, "Gianna" : 8000, "Alberto": 4000}

print(dict_double(d))
```

```
{'Mario': 2000, 'Gianna': 16000, 'Alberto': 8000}
```

The Python function must be saved in file: `dict_double.py`

Test file: `test_dict_double.py`

5 Count zero rows in a matrix (1 point)

Write a function `mat_count_zero_rows(m)` that:

- takes as argument a matrix `m`, represented as a list of lists;
- returns the number of rows in `m` whose elements are all zeroes.

We represent an $r \times c$ matrix in Python as a list with length r, where each element is a list of length c. You may assume that the argument is well formed, with $r \geq 1$, $c \geq 1$, and all the lists representing rows contain `c` numeric elements.

```
m = [[0, 0, 0], [0, 1, 0], [2, 1, 1], [0, 0, 0]]
print(mat_count_zero_rows(m))
```

```
2
```

The Python function must be saved in file: `mat_count_zero_rows.py`

Test file: `test_mat_count_zero_rows.py`

6 Change small elements to zero (1 point)

Write a function `change_small_to_zero(values)` that:

- expects a non-empty list of numeric values as an argument;
- directly modifies the `values` list by replacing with zero each element of `values` strictly smaller than the average of all elements.

The function does not return anything. Changes must be made into the list passed as argument.

The Python function must be saved in file: `change_small_to_zero.py`

Test file: `test_change_small_to_zero.py`